

**LA CANTINA**  
**Object Design Document**  
**Versione 1.1**



|                                   |                  |
|-----------------------------------|------------------|
| Progetto: LA CANTINA              | Versione: 1.1    |
| Documento: Object Design Document | Data: 27/12/2025 |

---

**Coordinatore del progetto:**

| Nome              | Matricola  |
|-------------------|------------|
| GRAZIOSO Fabrizio | 0512122950 |

**Partecipanti:**

| Nome               | Matricola  |
|--------------------|------------|
| LIGUORI Alessandro | 0512119887 |
| CICALESE Gabriele  | 0512116443 |
| GRAZIOSO Fabrizio  | 0512122950 |

|                    |                                      |
|--------------------|--------------------------------------|
| <b>Scritto da:</b> | GRAZIOSO Fabrizio, CICALESE Gabriele |
|--------------------|--------------------------------------|

## Revision History

| Data       | Versione | Descrizione                                      | Autore                      |
|------------|----------|--------------------------------------------------|-----------------------------|
| 22/12/2025 | 1.0      | Scrittura documento                              | Grazioso, Cicalese, Liguori |
| 27/12/2025 | 1.1      | Aggiunta dei Package e delle Classi implementate | CICALESE Gabriele           |

## Sommario

|      |                                                |    |
|------|------------------------------------------------|----|
| 1.   | Introduzione .....                             | 4  |
| 1.1. | Object design trade-offs .....                 | 4  |
| 1.2. | Interface documentation guidelines .....       | 4  |
| 1.3. | Definitions, acronyms, and abbreviations ..... | 5  |
| 1.4. | References .....                               | 5  |
| 2.   | Src .....                                      | 5  |
| 2.1. | it.unisa.lacantina.model.domain .....          | 5  |
| 2.2. | It.unisa.lacantina.model.dao .....             | 5  |
| 2.3. | It.unisa.lacantina.model.service .....         | 5  |
| 2.4  | It.unisa.lacantina.controller .....            | 6  |
| 3.   | Package .....                                  | 6  |
| 4.   | Class Interfaces .....                         | 9  |
| 4.1. | Sottosistema UserManagement .....              | 9  |
|      | <b>Class: UtenteDAO</b> .....                  | 9  |
| 4.2. | Sottosistema CatalogManagement .....           | 10 |
|      | <b>Class: ProdottoDao</b> .....                | 10 |
| 4.3. | Sottosistema OrderManagement .....             | 13 |
|      | <b>Class: RigaOrdineDao</b> .....              | 13 |
|      | <b>Class: OrdineDao</b> .....                  | 15 |
|      | <b>Class: CarrelloService</b> .....            | 17 |

# 1. Introduzione

## 1.1. Object design trade-offs

Durante la definizione del design degli oggetti per il sistema "La Cantina", il team di sviluppo ha dovuto affrontare diverse decisioni architetturali (trade-offs) per bilanciare le prestazioni, la consistenza dei dati e la manutenibilità, in linea con i *Design Goals* definiti nel SDD

- **Buy vs. Build:** Si è optato per un approccio **Build** (sviluppo da zero) utilizzando tecnologie standard Java EE (Servlet/JSP) invece di utilizzare piattaforme e-commerce precostituite. Questa scelta è dettata dalla necessità didattica di avere il controllo completo sul codice e di implementare logiche specifiche per la "filiera corta" e il km0.
- **Memory Space vs. Response Time (Gestione Carrello):** Per la gestione del carrello (Cart), si è scelto di mantenere lo stato interamente in memoria all'interno della HttpSession del server, senza persistenza immediata sul database
  - *Trade-off:* Questa scelta sacrifica l'occupazione di memoria RAM sul server (aumentando il carico per utente attivo) per garantire tempi di risposta (Response Time) immediati durante la navigazione e l'aggiunta di prodotti, evitando continue query di scrittura al DB prima del checkout
- **Data Access Strategy (JDBC vs ORM):** È stato scelto l'utilizzo di JDBC nativo con PreparedStatement invece di framework ORM (come Hibernate)
  - *Trade-off:* Sebbene questo comporti una maggiore verbosità nel codice dei DAO, garantisce un controllo granulare sulle query SQL, ottimizzando le performance e la sicurezza (prevenzione SQL Injection), riducendo l'overhead di astrazione.

## 1.2. Interface documentation guidelines

Per garantire la coerenza tra i moduli sviluppati dai diversi membri del team, sono state stabilite le seguenti convenzioni di codifica e documentazione delle interfacce:

- **Nomenclatura Classi:** Le classi devono essere nominate utilizzando sostantivi al singolare in stile PascalCase (es. Prodotto, Ordine), mantenendo la lingua italiana per coerenza con il dominio
- **Nomenclatura Metodi:** I metodi devono utilizzare verbi o frasi verbali in stile camelCase.
  - I metodi dei DAO devono seguire il suffisso standard DAO (es. UserDao, ProdottoDAO, OrdineDAO).
  - I metodi dei Controller devono riflettere l'azione (es. processCheckout, login).
- **Gestione Errori:** Lo stato di errore non deve essere restituito tramite valori interi, ma sollevando eccezioni specifiche. Le eccezioni SQL (SQLException) devono essere catturate e rilanciate come eccezioni applicative (es. RuntimeException o custom DAOException) per non esporre dettagli del database al livello superiore.
- **Collezioni:** I metodi che restituiscono collezioni (es. List<Prodotto>) non devono mai restituire null, ma una lista vuota (isEmpty()) in assenza di risultati, per evitare NullPointerException.

- **Visibilità:** Tutti gli attributi delle classi Entity devono essere private. L'accesso deve avvenire esclusivamente tramite metodi pubblici get e set.

### 1.3. *Definitions, acronyms, and abbreviations*

- **DAO (Data Access Object):** Pattern utilizzato per astrarre e incapsulare l'accesso ai dati persistenti.
- **MVC (Model-View-Controller):** Pattern architetturale su cui si basa l'intero sistema.
- **Entity:** Oggetto che rappresenta un dato persistente nel dominio (es. Utente, Prodotto)
- **Session Bean:** Oggetto mantenuto nella sessione utente (es. Carrello)
- **SDD:** System Design Document.
- **RAD:** Requirements Analysis Document.

### 1.4. *References*

- *Requirements Analysis Document (RAD) v2.1* - La Cantina.
- *System Design Document (SDD) v2.0* - La Cantina.

## 2. **Src**

All'interno delle cartelle del progetto è presente una sottocartella Database contenente il codice sql mentre un'altra sottocartella IMG contiene tutti i file multimediali utilizzati. Inoltre, il codice sorgente è organizzato in package Java che riflettono la decomposizione in sottosistemi definita nel SDD e l'architettura MVC.

### 2.1. *it.unisa.lacantina.model.domain*

Contiene tutti gli oggetti del dominio applicativo descritte sotto forma di classi, tali oggetti fanno riferimento alle entità presenti nel database e memorizzate all'interno della sessione utente come per la classe "Carrello". Tali classi **rappresentano i dati del dominio** e non contengono logica di business complessa ma solo attributi e metodi di accesso.

- **Classi implementate:** Utente, Prodotto, Ordine, RigaOrdine, Carrello, AziendaFornitrice.
- **Dipendenze:** Nessuna dipendenza verso altri package del sistema.

### 2.2. *It.unisa.lacantina.model.dao*

Contiene le classi che implementano il pattern **DAO**. Gestisce la comunicazione diretta con il DBMS MySQL tramite JDBC. Questo package corrisponde al livello "Persistent data management".

- **Classi implementate:** UtenteDAO, ProdottoDAO, OrdineDAO, RigaOrdineDAO, CarrelloDAO.
- **Dipendenze:** it.unisa.lacantina.model (utilizza le entity per input/output), java.sql, java.util.

### 2.3. *It.unisa.lacantina.model.service*

Contiene la classe CarrelloService che implementa i metodi per il comportamento del carrello salvato come variabile di sessione.

**Classi implementate:** CarrelloService.

- **Dipendenze:** it.unisa.lacantina.model, java.util.

## 2.4 *It.unisa.lacantina.controller*

Contiene le **Servlet** che fungono da Controller. Questo package gestisce la logica applicativa, riceve gli input dalla View (JSP), interagisce con i DAO, e determina la pagina di risposta con l'esito dell'operazione eseguita dall'utente.

- **Classi implementate:** AddToCartServlet, AdminAddStockServlet, AdminDeleteProdottoServlet, AdminInsertNewProductServlet, AdminRemoveStockServlet, CheckOutServlet, LoginServlet, LogoutServlet, ModifyOrderDataServlet, OrderNowServlet, QuantityIncDecServlet, RegisterServlet, RemoveFromCartServlet
- **Dipendenze:** it.unisa.lacantina.model, it.unisa.lacantina.model.dao, it.unisa.lacantina.util

## 2.5 *it.unisa.lacantina.util*

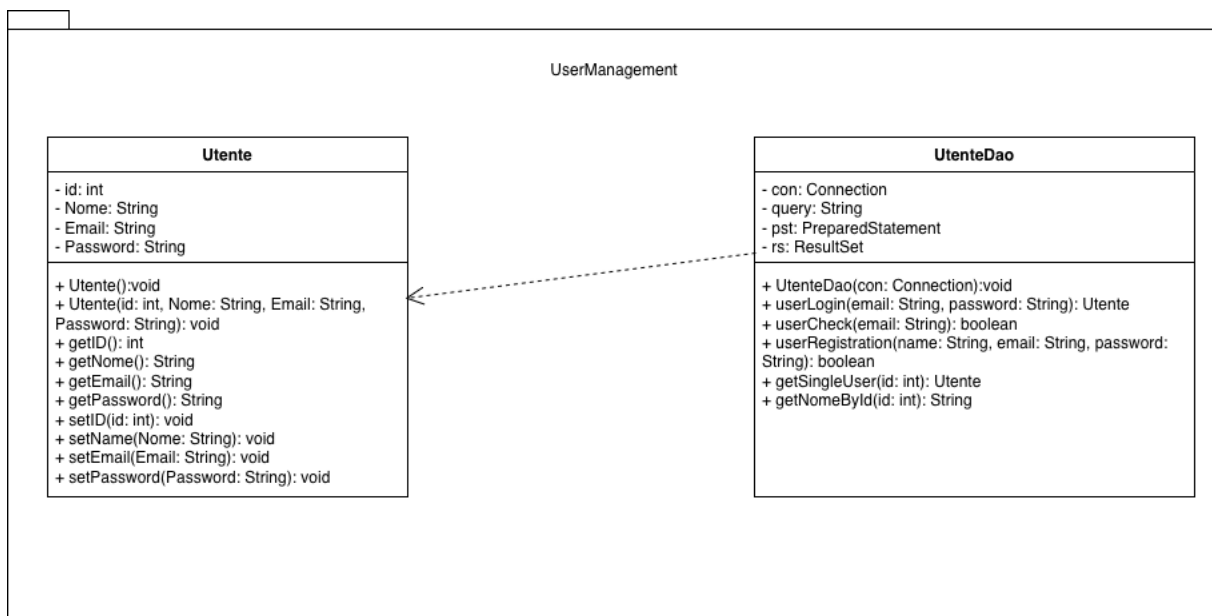
Contiene unicamente la classe Java *ConnectToDB* che sfrutta *JDBC* per effettuare la connessione al database impostando i parametri di root.

- **Classi implementate:** ConnectToDB
- **Dipendenze:** java.sql

# 3. Package

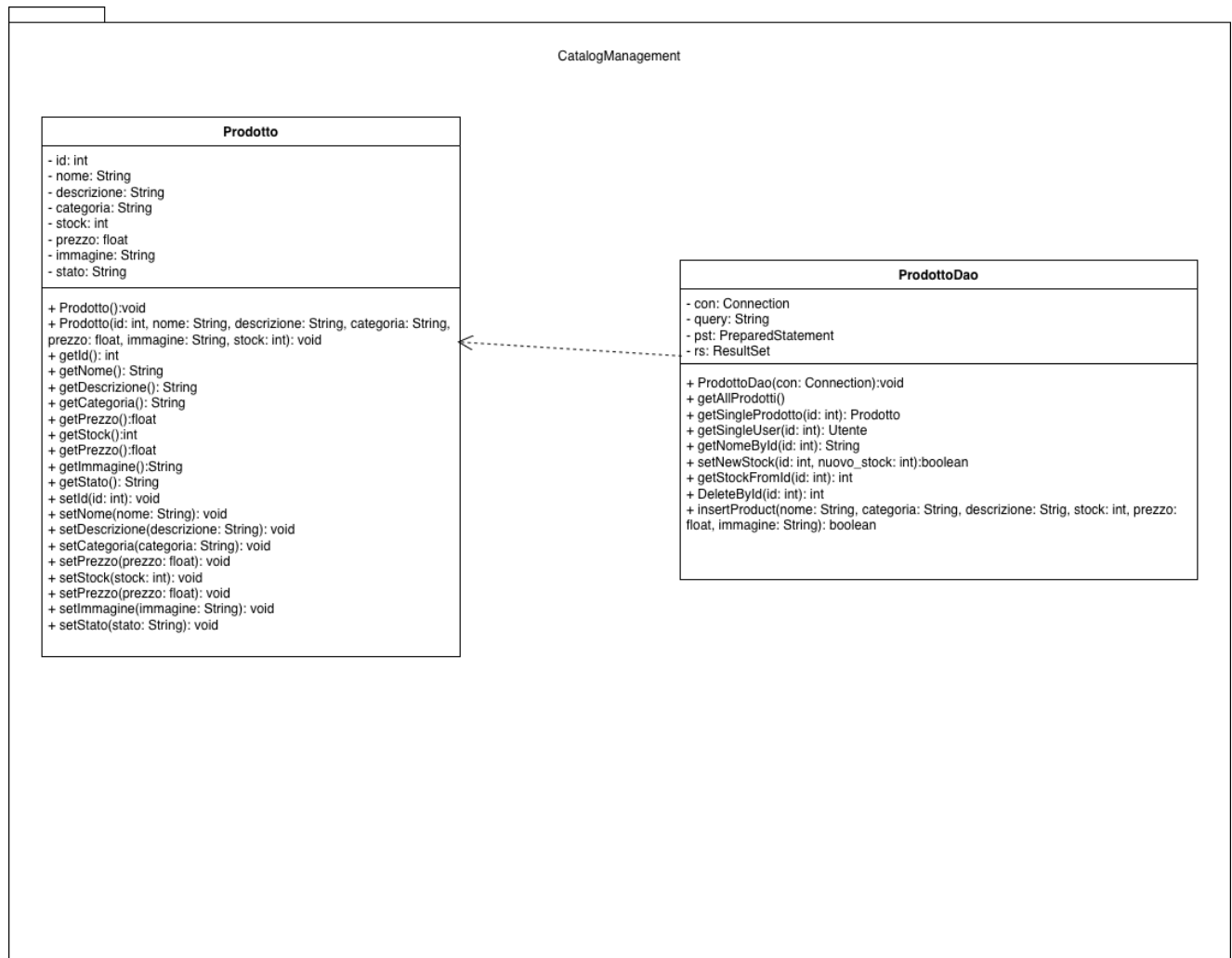
## 3.1 UserManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla gestione dell'utente**.



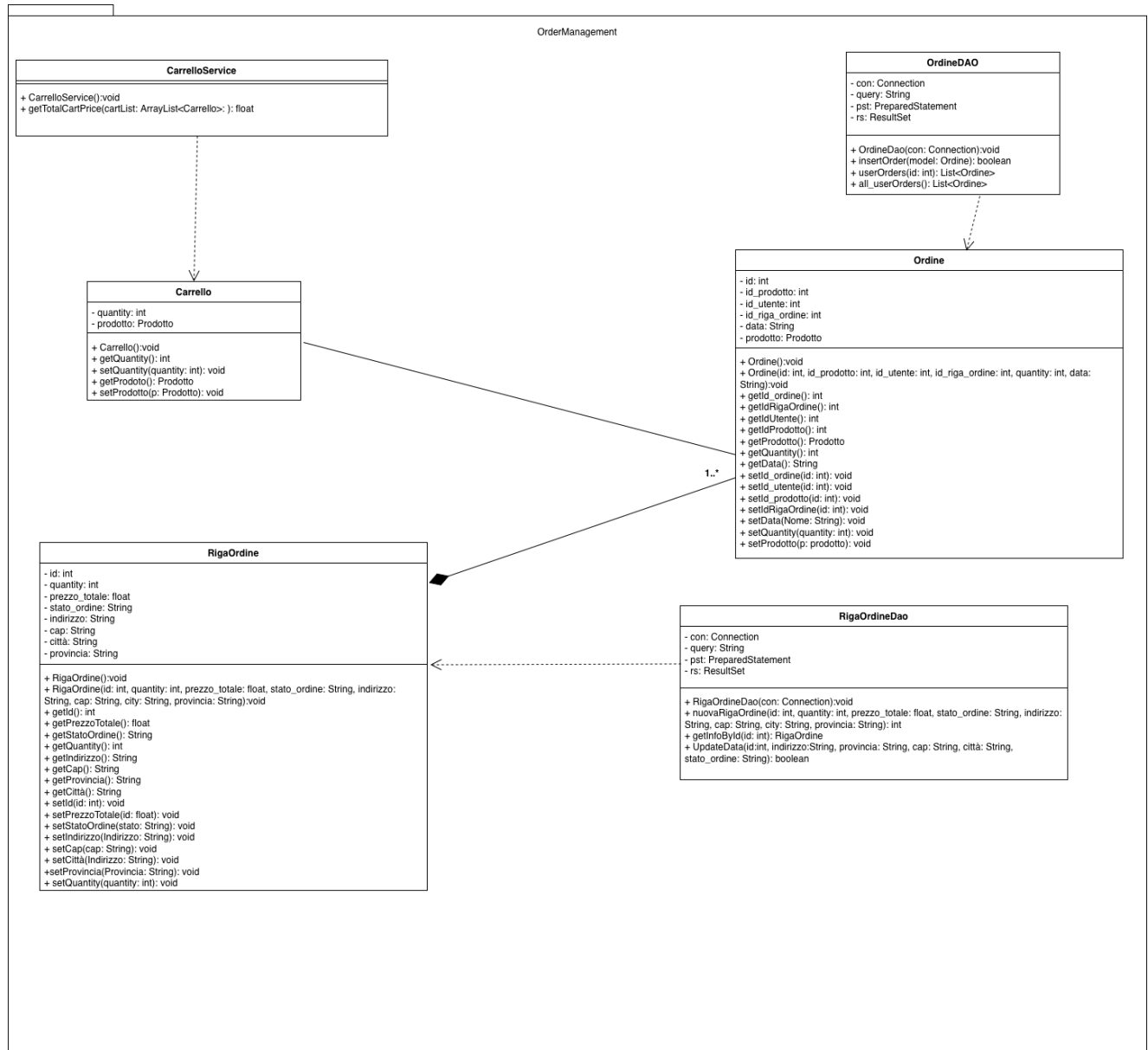
### 3.2 CatalogManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla visualizzazione dei prodotti all'interno dello shop del sito.**



### 3.3 OrderManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla gestione degli acquisiti dei clienti con il ServiceCarrello** per la gestione del carrello.



## 4. Class Interfaces

Questa sezione descrive le interfacce pubbliche delle classi chiave per i sottosistemi da implementare. **Per tutte le classi vale la regola che la connessione al database sia effettuata e che quindi la variabile di sessione con non sia null, ossia, pre: self.con <> null.**

### 4.1. Sottosistema UserManagement

#### Class: UtenteDAO

| UtenteDao          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrizione Classe | Questa classe si interfaccia con il DB, permettendo di gestire le operazioni relative all'account utente                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Pre-condizioni     | <p>Context UtenteDao::userRegistration(nome:String, email: String, password: String)<br/>Pre: nome &lt;&gt; null and email &lt;&gt; null and Utente.allInstances()-&gt;forAll(u   u.email &lt;&gt; email) and password &lt;&gt; null</p> <p>Context UtenteDao::userLogin(email: String, password: String)<br/>Pre: email &lt;&gt; null and password &lt;&gt; null</p> <p>Context UtenteDao::getNomeById(id: Integer)<br/>Pre: id &gt; 0 and Utente.allInstances()-&gt;exists(u   u.id = id)</p> <p>Context UtenteDao::getSingleUser(id: Integer)<br/>Pre: id &gt; 0 and Utente.allInstances()-&gt;exists(u   u.id = id)</p> <p>Context UtenteDao::UserCheck (email: String)<br/>Pre: email &lt;&gt; null</p> |
| Post-condizioni    | <p>Context UtenteDao::userRegistration(nome:String, email: String, password: String)<br/>Post: Utente.allInstances()-&gt;exists(u   u.nome = nome and u.email = email and u.password = password)</p> <p>Context UtenteDao::userLogin(email: String, password: String)<br/>Post: result &lt;&gt; null implies Utente.allInstances()-&gt;exists(u   u.email = email and u.password = password)<br/>and (result = null) implies Utente.allInstances()-&gt;forAll(u.email &lt;&gt; email or u.password &lt;&gt; password)</p> <p>Context UtenteDao::getNomeById(id: Integer)<br/>Post: result = Utente.allInstances()-&gt;any(u   u.id = id).nome</p>                                                            |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | <p>Context UtenteDao::getSingleUser(id: Integer)<br/> Post: (result<math>\neq</math>null implies Utente.allInstances()-&gt;exists(u   u.id = id and u = result)) and<br/> (result = null implies Utente.allInstances()-&gt;forall(u  u.id <math>\neq</math> id))</p> <p>Context UtenteDao::UserCheck (email: String)<br/> Post: (result=true implies Utente.allInstances()-&gt;exists(u  u.email =email))<br/> and (result = false implies Utente.allInstances()-&gt;forall(u  u.email <math>\neq</math>email))</p> |
| Invarianti | <p>Context Utente<br/> inv: id&gt;0<br/> inv: email<math>\neq</math>null<br/> inv:password<math>\neq</math>null</p>                                                                                                                                                                                                                                                                                                                                                                                                 |

## 4.2. Sottosistema CatalogManagement

### Class: ProdottoDao

| ProdottoDao        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrizione Classe | Questa classe si interfaccia con il DB, permettendo di gestire i prodotti registrati sul DB.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Pre-condizioni     | <p>Context ProdottoDao::getAllProdotti()<br/> Pre: true</p> <p>Context ProdottoDao::getSingleProdotto(id: Integer)<br/> Pre: id &gt; 0 and Prodotto.allInstances()-&gt;exists(p   p.id = id)</p> <p>Context ProdottoDao::setNewStock(id: Integer, stock: Integer)<br/> Pre: stock<math>\geq</math>0 and id&gt;0 and Prodotto.allInstances()-&gt;exists(p  p.id = id)</p> <p>Context ProdottoDao::getStockFromId(id: Integer)<br/> Pre: id&gt;0 and Prodotto.allInstances()-&gt;exists(p  p.id = id)</p> <p>Context ProdottoDao::DeleteById(id: Integer)<br/> Pre: id&gt;0 and Prodotto.allInstances()-&gt;exists(p  p.id = id)</p> |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | <p>Context ProdottoDao::InsertProduct(nome: String, categoria: String, descrizione: String, immagine: String, stock: Integer, prezzo: Real)</p> <p>Pre: nome <math>\diamond</math> null and categoria <math>\diamond</math> null and descrizione <math>\diamond</math> null and immagine <math>\diamond</math> null</p> <p>and stock <math>\geq</math> 0</p> <p>and prezzo <math>\geq</math> 0</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Post-condizioni | <p>Context ProdottoDao::getAllProdotti()</p> <p>Post: result-&gt;forAll(p  </p> <p>    p.id &gt; 0 and</p> <p>    p.nome <math>\diamond</math> null and</p> <p>    p.categoria <math>\diamond</math> null and</p> <p>    p.descrizione <math>\diamond</math> null and</p> <p>    p.immagine <math>\diamond</math> null and</p> <p>    p.prezzo <math>\geq</math> 0 and</p> <p>    p.stock <math>\geq</math> 0 and</p> <p>    p.stato <math>\diamond</math> null</p> <p>)</p> <p>Context ProdottoDao::getSingleProdotto(id: Integer)</p> <p>Post: (result <math>\diamond</math> null implies</p> <p>    result.id &gt; 0 and</p> <p>    result.nome <math>\diamond</math> null and</p> <p>    result.categoria <math>\diamond</math> null and</p> <p>    result.descrizione <math>\diamond</math> null and</p> <p>    result.immagine <math>\diamond</math> null and</p> <p>    result.prezzo <math>\geq</math> 0 and</p> <p>    result.stock <math>\geq</math> 0 and</p> <p>    result.stato <math>\diamond</math> null</p> <p>Context ProdottoDao::setNewStock(id: Integer, stock: Integer)</p> <p>Post: (result = true implies</p> <p>    Prodotto.allInstances()-&gt;exists(p   p.id = id and p.stock = newStock and p.stock <math>\geq</math> 0))</p> <p>and</p> <p>(result = false implies</p> <p>    Prodotto.allInstances()-&gt;forAll(p   p.id <math>\diamond</math> id or p.stock <math>\geq</math> 0))</p> |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | <p>Context ProdottoDao::getStockFromId(id: Integer)<br/> Post: result <math>\geq 0</math> and<br/> (Productto.allInstances()-&gt;exists(p   p.id = id) implies<br/> result = Prodotto.allInstances()-&gt;any (p   p.id = id).stock)</p> <p>Context ProdottoDao::DeleteById(id: Integer)<br/> Post: (result = true implies Prodotto.allInstances()-&gt;forall(p   p.id <math>\neq</math> id))</p> <p>Context ProdottoDao::insertProduct(nome: String, categoria: String,<br/> descrizione: String, immagine: String, stock: Integer, prezzo: Real)<br/> Post: result = true implies<br/> Prodotto.allInstances()-&gt;exists<br/> (p   p.nome = nome and<br/> p.categoria = categoria and<br/> p.descrizione = descrizione and<br/> p.immagine = immagine and<br/> p.stock = stock and<br/> p.prezzo = prezzo and<br/> p.stato = 'attivo')</p> |
| Invarianti | <p>Context Prodotto</p> <p>inv: id <math>&gt; 0</math><br/> inv: nome <math>\neq</math> null<br/> inv: categoria <math>\neq</math> null<br/> inv: descrizione <math>\neq</math> null<br/> inv: immagine <math>\neq</math> null<br/> inv: prezzo <math>\geq 0</math><br/> inv: stock <math>\geq 0</math><br/> inv: stato <math>\neq</math> null</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

### 4.3. Sottosistema OrderManagement

#### Class: RigaOrdineDao

| RigaOrdineDao      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrizione Classe | Questa classe si interfaccia con il DB, permettendo di inserire una nuova riga ordine, ossia, un nuovo contenitore logico di acquisto che raggruppa tutti gli ordini effettuati da un utente in una singola operazione di acquisto.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Pre-condizioni     | <p>Context:RigaOrdineDao::nuovaRigaOrdine(indirizzo: String, cap: String, provincia: String, prezzo_totale: Real, numero_ordini: Integer, String citta)</p> <p>Pre: indirizzo <math>\diamond</math> null and<br/>cap <math>\diamond</math> null and<br/>citta <math>\diamond</math> null and<br/>provincia <math>\diamond</math> null and<br/>prezzo_totale <math>&gt; 0</math> and<br/>numero_ordini <math>&gt; 0</math></p> <p>Context:RigaOrdineDao::getInfoById(id: Integer)</p> <p>Pre: id <math>&gt; 0</math></p> <p>Context:RigaOrdineDao::UpdateData(id: Integer )</p> <p>Pre: id <math>&gt; 0</math> and<br/>RigaOrdine.allInstances()-&gt;exists(r   r.id = id) and<br/>indirizzo <math>\diamond</math> null and<br/>provincia <math>\diamond</math> null and<br/>cap <math>\diamond</math> null and<br/>citta <math>\diamond</math> null and<br/>stato_ordine <math>\diamond</math> null</p> |
| Post-condizioni    | <p>Context:RigaOrdineDao::nuovaRigaOrdine(indirizzo: String, cap: String, provincia: String, prezzo_totale: Real, numero_ordini: Integer, String citta)</p> <p>Post: result <math>&gt; 0</math> implies<br/>RigaOrdine.allInstances()-&gt;exists(r  <br/>r.id = result and<br/>r.indirizzo = indirizzo and<br/>r.cap = cap and<br/>r.citta = citta and<br/>r.provincia = provincia and<br/>r.prezzo_totale = prezzo_totale and</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | <pre> r.numero_ordini = numero_ordini and r.stato_ordine = "attesa di conferma"  Context:RigaOrdineDao::getInfoById(id: Integer) Post: (result &lt;&gt; null implies   RigaOrdine.allInstances()-&gt;exists(r       r.id = id and     result.id = r.id and     result.numero_ordini = r.numero_ordini and     result.prezzo_totale = r.prezzo_totale and     result.stato_ordine = r.stato_ordine and     result.indirizzo = r.indirizzo and     result.cap = r.cap and     result.citta = r.citta and     result.provincia = r.provincia   ) ) and (result = null implies   RigaOrdine.allInstances()-&gt;forall(r   r.id &lt;&gt; id) )  Context:RigaOrdineDao::UpdateData(id: Integer) Post: result = true implies   RigaOrdine.allInstances()-&gt;exists(r       r.id = id and     r.indirizzo = indirizzo and     r.provincia = provincia and     r.cap = cap and     r.citta = citta and     r.stato_ordine = stato_ordine   ) </pre> |
| Invarianti | <pre> Context RigaOrdine inv: indirizzo &lt;&gt; null inv: cap &lt;&gt; null inv: citta &lt;&gt; null inv: provincia &lt;&gt; null inv: stato_ordine &lt;&gt; null inv: prezzo_totale &lt;&gt; null </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

|  |                                                              |
|--|--------------------------------------------------------------|
|  | inv: prezzo_totale > 0.0<br>inv: quantity > 0<br>inv: id > 0 |
|--|--------------------------------------------------------------|

### Class: OrdineDao

| OrdineDao          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrizione Classe | Questa classe si interfaccia con il DB, permettendo di effettuare acquisti sia istantanei che da carrello e registrarli sul database.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Pre-condizioni     | <p>Context: OrdineDao::insertOrder(model: Ordine)<br/>         Pre: model <math>\neq</math> null<br/>             and model.idProdotto &gt; 0<br/>             and Prodotto.allInstances()-&gt;exists(p   p.id = model.idProdotto)<br/>             and model.idUtente &gt; 0<br/>             and model.quantity &gt; 0<br/>             and model.data <math>\neq</math> null<br/>             and model.idRigaOrdine &gt; 0</p> <p>Context: OrdineDao::userOrders(id: Integer)<br/>         Pre: id &gt; 0 and Utente.allInstances()-&gt;exists(u   u.id = id)</p> <p>Context: OrdineDao::all_userOrders()<br/>         Pre: true</p>                                                                                                                                                         |
| Post-condizioni    | <p>Context: OrdineDao::insertOrder(model: Ordine)<br/>         Post: result = true implies<br/>             Ordine.allInstances()-&gt;exists(o  <br/>                 o.idProdotto = model.idProdotto and<br/>                 o.idUtente = model.idUtente and<br/>                 o.quantity = model.quantity and<br/>                 o.data = model.data and<br/>                 o.idRigaOrdine = model.idRigaOrdine and<br/>                 o.prodotto = model.prodotto)</p> <p>Context: OrdineDao::userOrders(id: Integer)<br/>         Post: post: result-&gt;forAll(o  <br/>             o.idUtente = id and<br/>             o.id &gt; 0 and<br/>             o.quantity &gt; 0 and<br/>             o.data <math>\neq</math> null and<br/>             o.idRigaOrdine &gt; 0 and</p> |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | <pre> o.prodotto &lt;&gt; null and o.prodotto.id &gt; 0 and o.prodotto.nome &lt;&gt; null and o.prodotto.categoria &lt;&gt; null and o.prodotto.descrizione &lt;&gt; null and o.prodotto.immagine &lt;&gt; null and o.prodotto.prezzo &gt;= 0 and o.prodotto.stock &gt;= 0 and o.prodotto.stato &lt;&gt; null )  Context:OrdineDao::all_userOrders() Post: context OrdineDao::all_userOrders() post: result-&gt;forAll(o       o.id &gt; 0 and     o.idUtente &gt; 0 and     o.idRigaOrdine &gt; 0 and     o.quantity &gt; 0 and     o.data &lt;&gt; null and     o.prodotto &lt;&gt; null and     o.prodotto.id &gt; 0 and     o.prodotto.nome &lt;&gt; null and     o.prodotto.categoria &lt;&gt; null and     o.prodotto.descrizione &lt;&gt; null and     o.prodotto.immagine &lt;&gt; null and     o.prodotto.prezzo &gt;= 0 and     o.prodotto.stock &gt;= 0 and     o.prodotto.stato &lt;&gt; null and     o.utente &lt;&gt; null and     o.utente.id &gt; 0 and     o.utente.nome &lt;&gt; null and     o.utente.email &lt;&gt; null and     o.utente.password &lt;&gt; null ) </pre> |
| Invarianti | <pre> context Ordine inv: id &gt; 0 inv: id_prodotto &gt; 0 inv: id_utente &gt; 0 inv: quantity &gt; 0 inv: data &lt;&gt; null inv: id_riga_ordine &gt; 0 inv: prodotto &lt;&gt; null inv: prodotto.id = prodotto.id </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

**Class: CarrelloService**

| CarrelloService    |                                                                                                                                                                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrizione Classe | La classe CarrelloService gestisce la logica di manipolazione del carrello degli utenti. Fornisce il metodo per calcolare il prezzo totale dei prodotti aggiunti a carrello. I dati sono <b>temporanei e non persistenti</b> , memorizzati nella sessione dell'utente durante la navigazione. |
| Pre-condizioni     | Context CarrelloService::getTotalCartPrice(cartList: Sequence (Carrello))<br>Pre:<br>cartList->forAll(c   c.prodotto <> null and c.prodotto.prezzo >= 0.0 and c.quantity > 0)                                                                                                                 |
| Post-condizioni    | Context CarrelloService::getTotalCartPrice(cartList: Sequence (Carrello))<br>Post:<br>result = cartList->iterate(c; sum: Real = 0.0   sum + c.prodotto.prezzo * c.quantity)                                                                                                                   |
| Invarianti         | context Carrello<br>inv: prodotto <> null<br>inv: prodotto.id > 0<br>inv: quantity > 0<br>inv: prodotto.prezzo > 0.0                                                                                                                                                                          |